

Mini project report
on

Title: Smart Campus Resource & Energy Analytics System

Under the guidance of
DR. UTKARSH MAHADEO KHAIRE

**DEPARTMENT OF DATA SCIENCE AND ARTIFICIAL
INTELLIGENCE
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY DHARWAD**

Certificate

This is to certify that the project, entitled *Smart Campus Resource & Energy Analytics System*, is a bonafide record of the Mini Project coursework presented by the students whose names are given below during 2025–2026 in partial fulfilment of the requirements of the degree of Bachelor of Technology in Computer Science and Engineering.

Roll No	Names of Students	
24BDS009	Rakhal Krishna	
24BDS010	Vineel Tej	
24BDS030	Rishikesh	
24BDS039	Arjun	

Smart Campus Resource & Energy Analytics System

Contents

1 Entity-Relationship Model	5
1.1 Entity Classification	5
1.2 Relationship Types	5
2 Complete Table Reference	5
2.1 Core Entities	5
2.2 Derived Entities (OLAP / Warehouse)	6
3 Normalization Analysis	7
3.1 First Normal Form (1NF)	7
3.2 Second Normal Form (2NF)	8
3.3 Third Normal Form (3NF)	9
3.4 Boyce-Codd Normal Form (BCNF)	9
4 Key Concepts Demonstrated	10
4.1 Functional Dependencies	10
4.2 Identifying vs Non-Identifying Relationships	10
4.3 Participation & Domain Constraints	11
5 Design Patterns Used	11
5.1 Stored Procedures & Triggers	11
5.2 Indexing Strategy	12
6 Anomaly Prevention	12
7 Transaction Management & Concurrency (Unit III)	13
7.1 ACID Properties	13
7.2 Isolation Levels & Deadlock	13
7.3 Two-Phase Locking (2PL)	14
8 Data Warehouse & ETL (Unit IV)	14
8.1 Star Schema & Snowflake Extension	14
8.2 OLAP Operations	15
8.3 ETL Pipeline	15
9 Full Stack Implementation	15
10 Challenges Faced & Solutions	16
Conclusion	16
References	17

Smart Campus Resource & Energy Analytics System Database Report

Entity-Relationship Model

The SmartCampus schema comprises 12 OLTP tables and 6 OLAP warehouse tables organized into functional categories: core entities, derived subtypes, junction tables, and analytical dimension/fact tables. Every relationship type taught in the DA264 curriculum is demonstrated within the schema.

1.1 Entity Classification

Type	Tables	Description
Strong	DEPARTMENT, USERS, BUILDING, EQUIPMENT_TYPE, TIME_SLOT	Independent entities with own PK; no existence dependency
Weak	MAINTENANCE_TICKET	Identity depends on ROOM context; identifying relationship with ROOM
Subtypes	STUDENT, FACULTY	Both inherit from USERS via user_id FK; IS-A generalization pattern
Associative	BOOKING, ACCESS_LOG, AUDIT_TRAIL	Resolve M:N relationships; carry relationship-specific attributes
Dependent	ENERGY_METER, ENERGY_READING, FLOOR, ROOM, EQUIPMENT	Surrogate PK but logically dependent on parent entity for meaning

1.2 Relationship Types

All cardinality types from the DA264 curriculum are present in the schema:

Relationship	Type	Implementation
DEPARTMENT ↔ FACULTY (head)	1:1 self-ref	dept.head_fac_id → FACULTY; circular FK added via ALTER TABLE
BUILDING ↔ FLOOR	1:N	FK floor.building_id; one building spans many floors
FLOOR ↔ ROOM	1:N	FK room.floor_id; one floor contains many rooms
ROOM ↔ ENERGY_METER	1:N	FK meter.room_id; one room has Electricity, Solar, Water meters
ENERGY_METER ↔ ENERGY_READING	1:N	FK reading.meter_id; each meter logs thousands of readings
USER ↔ BOOKING (via ROOM)	M:N	BOOKING resolves M:N; carries start/end time, purpose, status
STUDENT/FACULTY → USERS	IS-A	Generalization: USERS is supertype; subtypes share user_id PK/FK
ROOM → MAINTENANCE_TICKET	Weak entity	Identifying relationship; ticket meaning depends on room context

DEPARTMENT ↔ STUDENT & FACULTY	1:N (total)	Both student and faculty must belong to a department (NOT NULL FK)
--------------------------------	-------------	--

2. Complete Table Reference

This section details the schema for all 12 OLTP tables and 6 Data Warehouse tables in SmartCampusDB.

2.1 Core Entities

1. 'DEPARTMENT'

Attribute	Type & Constraints
<i>dept_id</i>	INT, PK
<i>dept_name</i>	VARCHAR(100), NOT NULL
<i>office_location</i>	VARCHAR(100)
<i>head_fac_id</i>	INT, NULL, FK → FACULTY (circular)

2. 'USERS'

Attribute	Type & Constraints
<i>user_id</i>	INT, PK
<i>name</i>	VARCHAR(100), NOT NULL
<i>email</i>	VARCHAR(100), NOT NULL, UNIQUE
<i>phone</i>	VARCHAR(20)
<i>user_type</i>	VARCHAR(20), CHECK IN ('Student','Faculty','Admin','Staff')

3. 'STUDENT'

Attribute	Type & Constraints
<i>student_id</i>	INT, PK
<i>user_id</i>	INT, NOT NULL, FK → USERS
<i>roll_no</i>	VARCHAR(20), NOT NULL, UNIQUE
<i>year_of_study</i>	INT, CHECK 1-5
<i>dept_id</i>	INT, NOT NULL, FK → DEPARTMENT

4. 'FACULTY'

Attribute	Type & Constraints
<i>faculty_id</i>	INT, PK
<i>user_id</i>	INT, NOT NULL, FK → USERS
<i>designation</i>	VARCHAR(100)
<i>employee_no</i>	VARCHAR(50), UNIQUE
<i>dept_id</i>	INT, NOT NULL, FK → DEPARTMENT

5. 'BUILDING'

Attribute	Type & Constraints
<i>building_id</i>	INT, PK

<i>bld_name</i>	VARCHAR(100), NOT NULL
<i>total_floors</i>	INT, CHECK >= 1
<i>bld_type</i>	VARCHAR(50)
<i>year_built</i>	INT
<i>dept_id</i>	INT, FK → DEPARTMENT

6. 'FLOOR'

Attribute	Type & Constraints
<i>floor_id</i>	INT, PK
<i>floor_no</i>	INT, NOT NULL
<i>floor_label</i>	VARCHAR(50)
<i>building_id</i>	INT, NOT NULL, FK → BUILDING

7. 'ROOM'

Attribute	Type & Constraints
<i>room_id</i>	INT, PK
<i>room_no</i>	VARCHAR(50), NOT NULL
<i>capacity</i>	INT, NOT NULL, CHECK > 0
<i>room_type</i>	VARCHAR(50)
<i>area_sqft</i>	INT, CHECK > 0
<i>floor_id</i>	INT, NOT NULL, FK → FLOOR

8. 'EQUIPMENT_TYPE'

Attribute	Type & Constraints
<i>type_id</i>	INT, PK
<i>type_name</i>	VARCHAR(100), NOT NULL
<i>power_kw</i>	FLOAT, CHECK >= 0
<i>manufacturer</i>	VARCHAR(100)

9. 'EQUIPMENT'

Attribute	Type & Constraints
<i>equip_id</i>	INT, PK
<i>serial_no</i>	VARCHAR(100), NOT NULL, UNIQUE
<i>purchase_date</i>	DATE
<i>status</i>	VARCHAR(50), CHECK IN ('Active','Inactive','Under Repair','Decommissioned')
<i>room_id</i>	INT, FK → ROOM
<i>type_id</i>	INT, NOT NULL, FK → EQUIPMENT_TYPE

10. 'ENERGY_METER'

Attribute	Type & Constraints
<i>meter_id</i>	INT, PK
<i>meter_type</i>	VARCHAR(50), CHECK IN ('Electricity','Solar','Water','Gas')
<i>install_date</i>	DATE
<i>is_active</i>	BIT, DEFAULT 1
<i>room_id</i>	INT, NOT NULL, FK → ROOM

11. 'ENERGY_READING'

Attribute	Type & Constraints
<i>reading_id</i>	INT, PK
<i>timestamp</i>	DATETIME, NOT NULL
<i>kwh_consumed</i>	FLOAT, NOT NULL, CHECK >= 0
<i>voltage</i>	FLOAT, CHECK > 0
<i>peak_flag</i>	BIT, DEFAULT 0
<i>meter_id</i>	INT, NOT NULL, FK → ENERGY_METER

12. 'BOOKING'

Attribute	Type & Constraints
<i>booking_id</i>	INT, PK
<i>start_time</i>	DATETIME, NOT NULL
<i>end_time</i>	DATETIME, NOT NULL
<i>purpose</i>	VARCHAR(200)
<i>status</i>	VARCHAR(50), CHECK IN ('Pending','Approved','Rejected','Cancelled','Completed')
<i>room_id</i>	INT, NOT NULL, FK → ROOM
<i>user_id</i>	INT, NOT NULL, FK → USERS
<i>slot_id</i>	INT, FK → TIME_SLOT

2.2 Derived Entities (OLAP / Warehouse)

Table	Purpose & Key Columns
fact_energy_consumption	Central fact table: fact_id PK, date_key FK, room_key FK, meter_key FK, kwh_consumed, voltage, peak_flag
dim_date	Time dimension: date_key PK, full_date, day_of_week, month_no, quarter, year, is_weekend — populated for full year 2025
dim_room	Denormalized room dimension (Star): room_key PK, room_id, room_no, room_type, capacity, floor_label, building_name, building_type, dept_name

dim_meter	Meter dimension: meter_key PK, meter_id, meter_type, room_no
dim_building (Snowflake)	Snowflake extension: normalizes building sub-dimension out of dim_room; building_key, building_id, bld_name, bld_type
dim_department (Snowflake)	Snowflake extension: dept_key, dept_id, dept_name, office_location — linked from dim_building

3. Normalization Analysis

All 12 OLTP tables in SmartCampusDB are normalized to Boyce-Codd Normal Form (BCNF). The analysis below walks through each normalization step using real tables from the project, showing exactly which functional dependencies were identified and how violations were eliminated. An interactive Normalization Dashboard in the React frontend visually demonstrates all four normal forms with FD arrows and before/after tables.

3.1 First Normal Form (1NF)

Rule

Every column must contain only atomic (indivisible) values. No repeating groups or multi-valued attributes in any cell.

Problem: Multi-Valued Equipment Column

Before normalization, if equipment types were stored directly in BOOKING, a row for a booked lab might look like:

booking_id	room_no	student_name	equipment_list ✕
1	L301	Arjun	PC, Projector, AC
2	L101	Priya	Oscilloscope, FPGA

The equipment_list column contains multiple values in one cell. This breaks atomicity — the 1NF violation.

Solution: Separate EQUIPMENT and EQUIPMENT_TYPE Tables

We decomposed the multi-valued attribute into two separate tables. Each equipment item gets its own row:

equip_id	serial_no	type_id (FK → EQUIPMENT_TYPE) ✓
1	PC001	1 (Desktop Computer)
2	PROJ01	3 (Projector)
3	AC001	4 (Air Conditioner)

Verdict1NF satisfied across all 12 tables. Every column stores one atomic value. All M:N relationships resolved via junction/associative tables.

3.2 Second Normal Form (2NF)

Rule

Must be in 1NF. Every non-key attribute must be fully functionally dependent on the entire primary key — not just part of it. Partial dependencies only exist when the primary key is composite.

Testing SmartCampusDB Tables

Table (PK Type)	Non-Key Attributes	Partial Dependency? Verdict
All 12 OLTP tables —	All non-key attributes	✓No — partial dependency cannot exist

Single-column surrogate INT PK		when PK is a single column. All tables use surrogate INT IDENTITY PKs.
BOOKING: references both room_id and user_id	start_time, end_time, purpose, status	✔No partial dependency — these attributes describe the booking event itself (which user booked which room at what time). Both FKs needed for context.

Verdict2NF satisfied. By design, all tables use single-column surrogate primary keys, making partial dependency structurally impossible.

3.3 Third Normal Form (3NF)

Rule

Must be in 2NF. No non-key attribute should transitively depend on the primary key through another non-key attribute (no $A \rightarrow B \rightarrow C$ where B is not a key).

The Transitive Dependency We Prevented

If we had stored building and department details directly in ENERGY_READING:

reading_id	meter_id	kwh	building_name	dept_name	
1	5	2.3	E Block	CSE	← WRONG
2	6	1.8	E Block	CSE	← Redundancy!

Here: reading_id → meter_id → room_id → floor_id → building_id → dept_name. The dept_name depends on the building, not on the reading itself. That is a transitive chain.

Our Fix: Full Hierarchy of Separate Tables

Violation Found	Resulting Anomaly	Fix Applied
dept_name stored with ENERGY_READING	Redundancy; update anomaly if dept renamed	Dept linked via Reading → Meter → Room → Floor → Building → Dept hierarchy
building_name, floor in BOOKING	Update anomaly if building renamed	Extract BUILDING, FLOOR, ROOM; BOOKING references only room_id FK
equip_type → manufacturer (transitive)	Deletion anomaly if last equipment of a type removed	Separate EQUIPMENT_TYPE table; EQUIPMENT references type_id FK

Verdict3NF satisfied. Transitive chains broken by the Room → Floor → Building → Department hierarchy and the EQUIPMENT_TYPE extraction.

3.4 Boyce-Codd Normal Form (BCNF)

Rule

For every non-trivial functional dependency $X \rightarrow Y$, X must be a superkey. BCNF is stricter than 3NF and eliminates anomalies even when there are overlapping candidate keys.

Proof: Sample Tables

Table	Functional Dependencies	BCNF Verdict
DEPARTMENT	$dept_id \rightarrow dept_name, office_location, head_fac_id$	$dept_id$ is PK (superkey) ✓
USERS	$user_id \rightarrow name, email, phone, user_type$ $email \rightarrow user_id$ (candidate key via UNIQUE)	Both $user_id$ and $email$ are candidate keys (superkeys) ✓
ENERGY_METER	$meter_id \rightarrow meter_type, install_date, is_active, room_id$	$meter_id$ is PK (superkey) ✓
ENERGY_READING	$reading_id \rightarrow timestamp, kwh_consumed, voltage, peak_flag, meter_id$	$reading_id$ is PK (superkey) ✓
EQUIPMENT	$equip_id \rightarrow serial_no, status, room_id, type_id$ $serial_no \rightarrow equip_id$ (candidate via UNIQUE)	Both $equip_id$ and $serial_no$ are superkeys ✓
BOOKING	$booking_id \rightarrow start_time, end_time, purpose, status, room_id, user_id$	$booking_id$ is PK (superkey) ✓

VerdictBCNF satisfied across all 12 OLTP tables. Every determinant is a candidate key. The interactive Normalization Dashboard in the frontend shows this for all tables with FD arrows.

4. Key Concepts Demonstrated

4.1 Functional Dependencies

Key functional dependencies documented in the schema (also written as comments above each CREATE TABLE in DBMS.sql):

```
-- DEPARTMENT
dept_id → dept_name, office_location, head_fac_id

-- USERS (2 candidate keys)
user_id → name, email, phone, user_type
email → user_id, name, phone, user_type    (UNIQUE constraint)

-- ENERGY READING
reading_id → timestamp, kwh_consumed, voltage, peak_flag, meter_id

-- BOOKING
booking_id → start_time, end_time, purpose, status, room_id, user_id
```

All non-key attributes depend on the primary key and nothing but the primary key — confirming BCNF compliance throughout.

4.2 Identifying vs Non-Identifying Relationships

Identifying: MAINTENANCE_TICKET → ROOM

ticket_id	room_id (FK)	Note
1	109	Ticket only has meaning in the context of room 109
2	104	Cannot exist or be understood without its room

The MAINTENANCE_TICKET's identity is tied to ROOM — this is the weak entity / identifying relationship pattern from Unit I.

Non-Identifying: BOOKING → ROOM

booking_id	room_id (FK)	Note
501	104	BOOKING has independent identity via its own surrogate PK
502	104	FK room_id is NOT part of BOOKING's PK

4.3 Participation & Domain Constraints

Constraint Type	Where Applied	Implementation
Total participation	STUDENT → DEPARTMENT	dept_id NOT NULL — every student must belong to a department

Domain constraint	USERS.user_type	CHECK (user_type IN ('Student','Faculty','Admin','Staff'))
Domain constraint	BOOKING.status	CHECK (status IN ('Pending','Approved','Rejected','Cancelled','Completed'))
Referential integrity	All FKs	SQL Server FOREIGN KEY constraints on all 26 relationships
Uniqueness	USERS.email, roll_no, employee_no	UNIQUE constraints prevent duplicate identifiers

5. Design Patterns Used

5.1 Stored Procedures & Triggers

SmartCampusDB implements encapsulated business logic via two stored procedures and three triggers:

Object	Type	Purpose
usp_AddBooking	Stored Procedure	Conflict-checking booking insert with BEGIN/COMMIT/ROLLBACK; raises error if room already booked for that window
usp_UpdateBookingStatus	Stored Procedure	Validates status transition and updates booking; audit trail inserted automatically by trigger
trg_booking_insert	AFTER INSERT Trigger	Auto-inserts AUDIT_TRAIL row whenever a new booking is created
trg_booking_update	AFTER UPDATE Trigger	Records old_status → new_status in AUDIT_TRAIL on every status change
trg_booking_nodelete	INSTEAD OF DELETE Trigger	Prevents direct deletion of Approved bookings; must set status to Cancelled first

5.2 Indexing Strategy

Index	Type	Query Served
idx_energy_timestamp on ENERGY_READING(meter_id, timestamp)	Composite B-Tree	Time-series energy analysis; most-queried join
idx_booking_conflict on BOOKING(room_id, start_time)	Composite B-Tree	Double-booking conflict detection in usp_AddBooking
idx_access_user on ACCESS_LOG(user_id)	B-Tree	User activity lookups and security audits
idx_active_meters on ENERGY_METER(is_active, meter_type)	Filtered/Partial	Dashboard queries for active meters only
idx_ticket_status on MAINTENANCE_TICKET(status, priority)	Composite B-Tree	Open/high-priority ticket queries

Primary clustered indexes exist on every table's PK (SQL Server IDENTITY column). The B-Tree structure means: leaf nodes store actual row data for clustered indexes, and PK pointers for non-clustered secondary indexes.

6. Anomaly Prevention

6.1 Insertion Anomalies — Prevented

Problem: In a denormalized design, you cannot add a building to the system unless at least one room exists in it.

Our design: BUILDING is an independent strong entity. You can insert a new building even before any floors or rooms are created. Similarly, EQUIPMENT_TYPE can exist before any equipment of that type is purchased.

6.2 Update Anomalies — Prevented

Problem: If building_name were stored in every BOOKING row, renaming 'E Block' to 'Engineering Block' would require updating hundreds of booking rows.

Our design: bld_name exists in exactly ONE place — the BUILDING table. All bookings reference room_id → floor_id → building_id. Change the name once; all queries automatically reflect it via JOIN.

6.3 Deletion Anomalies — Prevented

Problem: Deleting the last equipment of type 'Projector' would lose all knowledge that projectors exist in the campus.

Our design: EQUIPMENT_TYPE is independent. Deleting all projector EQUIPMENT rows leaves the 'Projector' row in EQUIPMENT_TYPE intact. Additionally, trg_booking_nodelete prevents accidental loss of approved booking records.

Action	Effect in SmartCampusDB
Delete last AC unit (equip_id=5) from EQUIPMENT	EQUIPMENT_TYPE row (type_id=4, 'Air Conditioner') survives ✓
Try to delete an Approved booking directly	trg_booking_nodelete raises error: 'Set status to Cancelled first' ✓

7. Transaction Management & Concurrency Control (Unit III)

All transaction demos are implemented in transactions_unit3_demo.sql and were validated live in SQL Server Management Studio. This section directly addresses CO3 and LO4 of DA264.

7.1 ACID Properties

Property	Scenario	How Demonstrated	Result
Atomicity	Booking INSERT + ACCESS_LOG INSERT as one unit; step 2 uses invalid room_id 9999	FK violation on step 2 fires ROLLBACK; both inserts undone	Booking 502 does NOT exist after failed transaction — verified with SELECT
Consistency	Try booking room already booked in same time window; try booking with end < start	usp_AddBooking rejects overlap; CHECK constraint chk_bk_times fires	No invalid data inserted; constraint violation messages confirm enforcement
Isolation	Session B SELECT on booking 501 while Session A holds open UPDATE transaction	Session B blocked for 6 min 57 sec under READ COMMITTED	Session B got committed value (Pending), not dirty Approved value — dirty read prevented
Durability	Booking 505 committed; SQL Server WAL ensures persistence	COMMIT issued; WAL writes log before acknowledging	Booking 505 confirmed with SELECT after commit; survives restart via WAL REDO

7.2 Isolation Levels & Deadlock

READ COMMITTED (SQL Server default)

- Session B's SELECT on booking_id=501 was blocked for 6 minutes 57 seconds while Session A held an open uncommitted UPDATE transaction
- Session B unblocked and returned the committed value the instant Session A executed COMMIT TRANSACTION
- This proves READ COMMITTED prevents dirty reads: a transaction only sees data that has been committed

Deadlock Scenario

A circular-wait deadlock was constructed using two SSMS windows:

Step	Window 1 (T1)	Window 2 (T2)
1	BEGIN TRAN; UPDATE BOOKING SET status='Approved' WHERE booking_id=501; WAITFOR DELAY '5s'	(waiting)
2	(sleeping 5 sec)	BEGIN TRAN; UPDATE BOOKING SET status='Cancelled' WHERE booking_id=502
3	UPDATE BOOKING SET status='Approved' WHERE booking_id=502 ← BLOCKED by T2	UPDATE BOOKING SET status='Cancelled' WHERE booking_id=501 ← BLOCKED by T1
Resu	SQL Server raises Msg 1205: 'Transaction was deadlocked on lock resources with another process'	

It	and has been chosen as the deadlock victim. Rerun the transaction.' — one transaction auto-rolled back, other completes.
-----------	--

7.3 Two-Phase Locking (2PL) Protocol

Demonstrated using explicit HOLDLOCK and UPDLOCK hints in SQL Server:

```
-- GROWING PHASE: acquire all needed locks
BEGIN TRANSACTION;
  SELECT booking_id FROM BOOKING WITH (HOLDLOCK) WHERE booking_id=501; -- S-lock
  SELECT booking_id FROM BOOKING WITH (UPDLOCK, HOLDLOCK) WHERE booking_id=502; -- U-lock

-- LOCK POINT: all locks held simultaneously

-- SHRINKING PHASE: modify then release all at once
  UPDATE BOOKING SET status='Approved' WHERE booking_id=502; -- U-lock → X-lock
COMMIT TRANSACTION; -- ALL locks released simultaneously
```

2PL Timeline: t1[Acquire S-lock] → t2[Acquire U-lock] → t3[Lock Point] → t4[Upgrade U→X, UPDATE] → t5[COMMIT, release all locks]. No new locks acquired after the lock point — strict 2PL guaranteed.

8. Data Warehouse & ETL (Unit IV)

8.1 Star Schema & Snowflake Extension

The central fact table captures every energy reading event. The Star Schema surrounds it with four dimension tables. A Snowflake extension normalizes the building and department sub-dimensions out of dim_room:

Table	Schema Role & Key Columns
fact_energy_consumption	FACT (center): fact_id IDENTITY PK, date_key, room_key, meter_key, kwh_consumed, voltage, peak_flag
dim_date	TIME dimension: date_key (YYYYMMDD), full_date, day_of_week, month_no, quarter, year, is_weekend — populated for all 365 days of 2025
dim_room	ROOM dimension (Star — denormalized): room_key, room_id, room_no, room_type, capacity, floor_label, building_name, building_type, dept_name all in one row
dim_meter	METER dimension: meter_key, meter_id, meter_type, room_no
dim_building	SNOWFLAKE extension: building_key, building_id, bld_name, bld_type, total_floors, year_built — dim_room references this
dim_department	SNOWFLAKE extension: dept_key, dept_id, dept_name, office_location — dim_building references this

8.2 OLAP Operations

OLAP Op.	Granularity	Query Description
ROLL-UP	Daily → Monthly → Quarterly	Energy aggregated upward through time hierarchy using GROUP BY ROLLUP. Subtotals generated automatically at each level.
DRILL-DOWN	Campus → Building → Floor → Room	Energy broken from campus total down to individual room via spatial hierarchy JOIN through dim_room → FLOOR → BUILDING.
SLICE	Year 2025 only	Single dimension filter: WHERE dim_date.year = 2025. Shows all rooms for that year only.
DICE	Q1 2025 + Computer Labs + Electricity	Multi-dimensional filter across time (Q1), room type (Computer Lab), and meter type (Electricity) simultaneously.
PIVOT	Buildings as columns, months as rows	Cross-tabulation using SQL Server PIVOT: each building becomes a column header; monthly kWh fills the grid.

8.3 ETL Pipeline

A Python-based incremental ETL pipeline (etl_pipeline.py) automates data flow from OLTP to the Data Warehouse:

- Watermark-based incremental load** — pulls only NEW ENERGY_READING rows with timestamp > MAX(full_date) in dim_date. First run loads all; subsequent runs only load deltas.
- IQR outlier detection** — Q1/Q3 computed on kwh_consumed; readings outside [Q1 – 1.5×IQR, Q3 + 1.5×IQR] discarded as sensor errors.

- **Surrogate key mapping** — OLTP room_id and meter_id mapped to warehouse room_key and meter_key via dimension table joins; date_key generated in YYYYMMDD integer format.
- **Bulk append** — pandas to_sql with if_exists='append' and index=False (fact_id is IDENTITY column). Logger reports row count loaded.

9. Full Stack Implementation

Layer	Technology	Key Features
OLTP Database	SQL Server 2022 (SSMS)	12 entities, BCNF normalized, 1000+ rows seed data, triggers, stored procedures
OLAP Warehouse	SQL Server (same instance, separate schema)	Star + Snowflake schemas, 5 OLAP queries, ETL pipeline
API Backend	Python FastAPI	JWT-based auth (HS256), role-based access (admin vs viewer), CORS for React
Frontend	React + Vite + Tailwind CSS	8 dashboard pages: Overview, Energy Analysis, Infrastructure, Bookings, Maintenance, Access, Data Warehouse, Normalization
Normalization Page	Interactive React Component	1NF/2NF/3NF/BCNF tabs with FD arrows, before/after tables, All Tables view — built directly from SmartCampusDB schema

10. Challenges Faced & Solutions

#	Challenge	Root Cause	Solution Applied
1	Circular FK between DEPARTMENT and FACULTY	<i>DEPARTMENT.head_fac_id references FACULTY, but FACULTY.dept_id references DEPARTMENT — chicken-and-egg</i>	Created DEPARTMENT first with head_fac_id = NULL, created FACULTY, then ALTER TABLE ADD CONSTRAINT to add the circular FK
2	Session B blocked for 7+ minutes during isolation demo	<i>Session A left an open transaction from earlier failed attempts; KILL command issued inside an open transaction failed</i>	Used sys.dm_exec_requests to identify blocking session ID 70; KILL 70 issued from a clean 3rd window
3	ETL Python script connection failure to SQL Server	<i>Server name contained backslash (rakhal\RSQL) not handled by urllib</i>	Used raw string r'rakhal\RSQL' and urllib.parse.quote_plus to encode the ODBC connection string properly
4	Frontend: 'Invalid object name BOOKING' in SSMS Window 2	<i>SSMS database dropdown was set to 'master' instead of SmartCampusDB</i>	Added USE SmartCampusDB at top of every script; changed SSMS connection default database in properties
5	Building Energy Roll-Up only showing 2 of 6 buildings	<i>MOCK data array in App.jsx only had E Block and PI Block hardcoded</i>	Added all 6 buildings (B Block, G Block, H Block, M Block) with realistic kWh values to the MOCK.buildings array

11. Entity-Relationship Diagram

The Entity-Relationship (ER) Diagram below provides a complete visual representation of the SmartCampusDB schema, covering all 12 OLTP operational tables and 6 OLAP Data Warehouse tables. Every cardinality type, entity category, and relationship pattern from the DA264 curriculum is demonstrated within the schema.

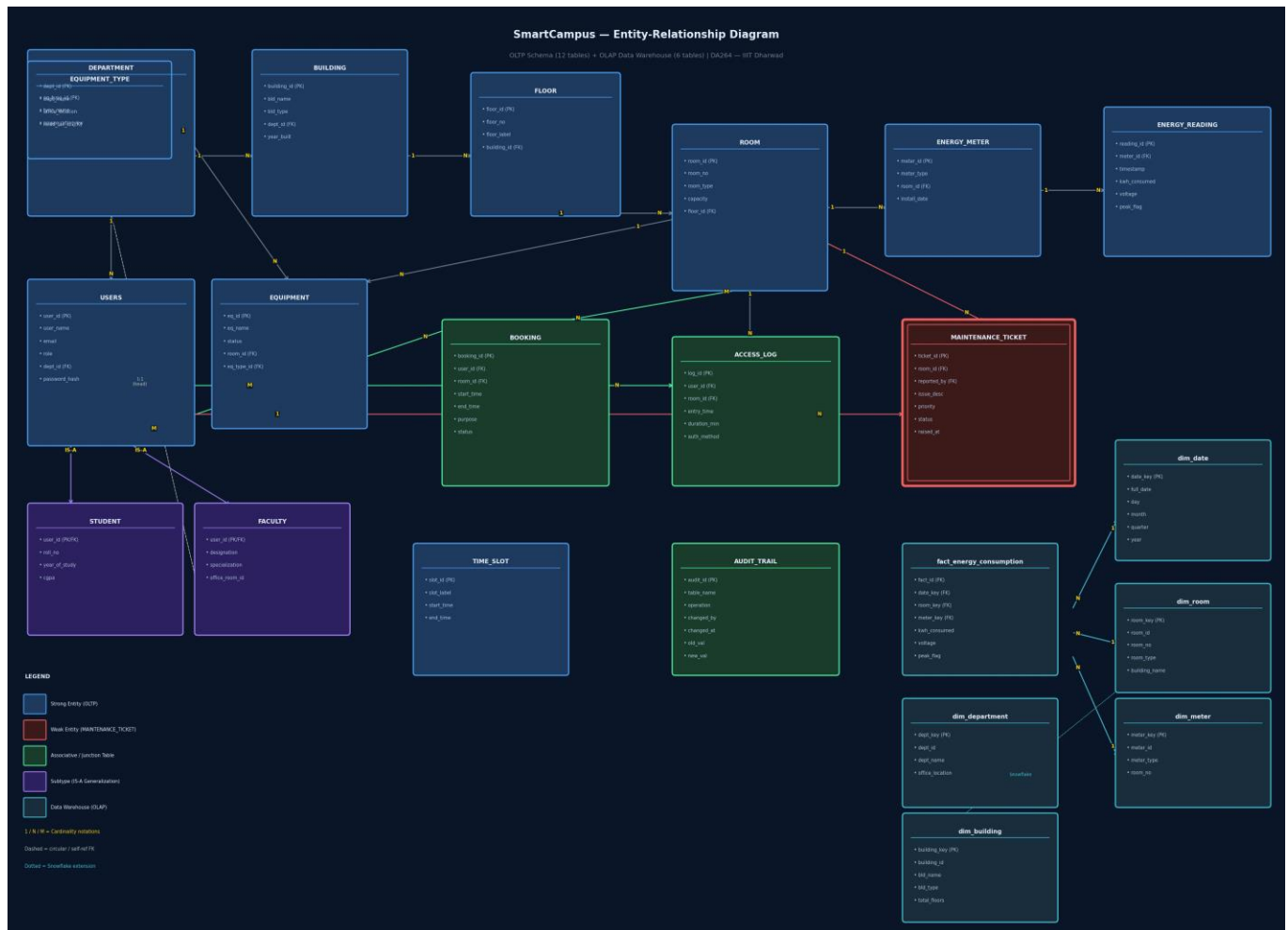


Figure 11.1 — SmartCampus Full ER Diagram (OLTP + OLAP)

11.1 Colour Conventions

Colour / Border	Entity Type	Tables
Dark blue, solid blue border	Strong Entity (OLTP)	DEPARTMENT, BUILDING, FLOOR, ROOM, ENERGY_METER, ENERGY_READING, USERS, EQUIPMENT_TYPE, TIME_SLOT
Dark red, double border	Weak Entity	MAINTENANCE_TICKET (identity depends on ROOM)
Purple, purple border	Subtype (IS-A)	STUDENT, FACULTY (inherit from USERS)
Dark green, teal border	Associative / Junction	BOOKING, ACCESS_LOG, AUDIT_TRAIL
Dark teal, teal border	OLAP Warehouse	fact_energy_consumption, dim_date, dim_room, dim_meter, dim_department, dim_building

11.2 ER Diagram Explanation

Spatial Hierarchy (top of diagram, left → right)

The campus physical structure flows as a five-level chain: **DEPARTMENT** → **BUILDING** (1:N) → **FLOOR** (1:N) → **ROOM** (1:N) → **ENERGY_METER** (1:N) → **ENERGY_READING** (1:N). This hierarchy eliminates all transitive dependencies — for example, room attributes cannot transitively depend on a building if a FLOOR intermediate table is present. Every step is enforced by a NOT NULL foreign key constraint.

Strong Entities

Dark-blue boxes are Strong Entities — they own an independent primary key and have no existence dependency on any other table. **DEPARTMENT**, **BUILDING**, **FLOOR**, **ROOM**, **ENERGY_METER**, **ENERGY_READING**, **USERS**, **EQUIPMENT_TYPE**, and **TIME_SLOT** all fall in this category. They form the structural backbone of the OLTP schema.

Weak Entity — MAINTENANCE_TICKET

MAINTENANCE_TICKET is drawn with a double border (dark red). Its logical identity is dependent on **ROOM** — a maintenance ticket is meaningless without the room it was raised for. The relationship between **ROOM** and **MAINTENANCE_TICKET** is therefore an *identifying relationship*. The `room_id` FK in **MAINTENANCE_TICKET** is NOT NULL and part of its identifying key.

IS-A Generalization — STUDENT and FACULTY

STUDENT and **FACULTY** (purple boxes) are subtypes that inherit from the **USERS** supertype via shared `user_id` (primary key AND foreign key simultaneously). **USERS** holds common attributes (`user_name`, `email`, `role`, `dept_id`, `password_hash`). **STUDENT** adds `roll_no`, `year_of_study`, and `cgpa`; **FACULTY** adds `designation`, `specialization`, and `office_room_id`. This IS-A pattern avoids NULL-heavy columns in a single merged table.

Associative Tables — M:N Resolution

BOOKING resolves the M:N between **USERS** and **ROOM**, carrying `start_time`, `end_time`, `purpose`, and `status` as relationship attributes. **ACCESS_LOG** resolves the M:N between **USERS** and **ROOM** for physical access events, carrying `entry_time`, `duration_min`, and `auth_method`. **AUDIT_TRAIL** is a system-wide associative table populated automatically by triggers on every DML operation, recording `table_name`, `operation`, `changed_by`, `changed_at`, and old/new values.

Circular 1:1 Self-Referential FK

A dashed arrow from **DEPARTMENT** back to **FACULTY** represents the `head_fac_id` column: every department has exactly one faculty head. Because **FACULTY**.`dept_id` also references **DEPARTMENT**, a circular FK dependency arises. This was resolved by first creating **DEPARTMENT** with `head_fac_id` = NULL, inserting all **FACULTY** rows, then issuing `ALTER TABLE ADD CONSTRAINT` to add the circular FK safely.

OLAP / Warehouse Layer (bottom-right)

The teal boxes form the Star Schema: **fact_energy_consumption** (central fact table, 515 rows) connects to four dimension tables via surrogate key foreign keys — `dim_date` (`date_key`, 365 rows), `dim_room` (`room_key`, 16 rows), `dim_meter` (`meter_key`, 27 rows), and a Snowflake extension comprising `dim_building` and `dim_department`. Dotted arrows denote the Snowflake normalisation: `dim_room` references `dim_building` which references `dim_department`, enabling Campus → Dept → Building → Floor → Room drill-down hierarchy in OLAP queries.

12. User Interface — Dashboard Screenshots & Explanations

The SmartCampus frontend is built with **React + Vite + Tailwind CSS** and communicates with a **FastAPI (Python)** backend via JWT-authenticated REST endpoints over port 8000. Nine navigational sections are accessible from the left sidebar: Overview, Energy Analysis, Infrastructure, Room Bookings, Maintenance, Access & Security, User Analytics, Data Warehouse, Live Operations, and Normalization. Each section queries specific SQL Server views, stored procedures, or OLAP fact tables and renders results as charts, tables, and KPI cards. All screenshots below were captured from a live Demo Mode session.

12.1 Overview Dashboard

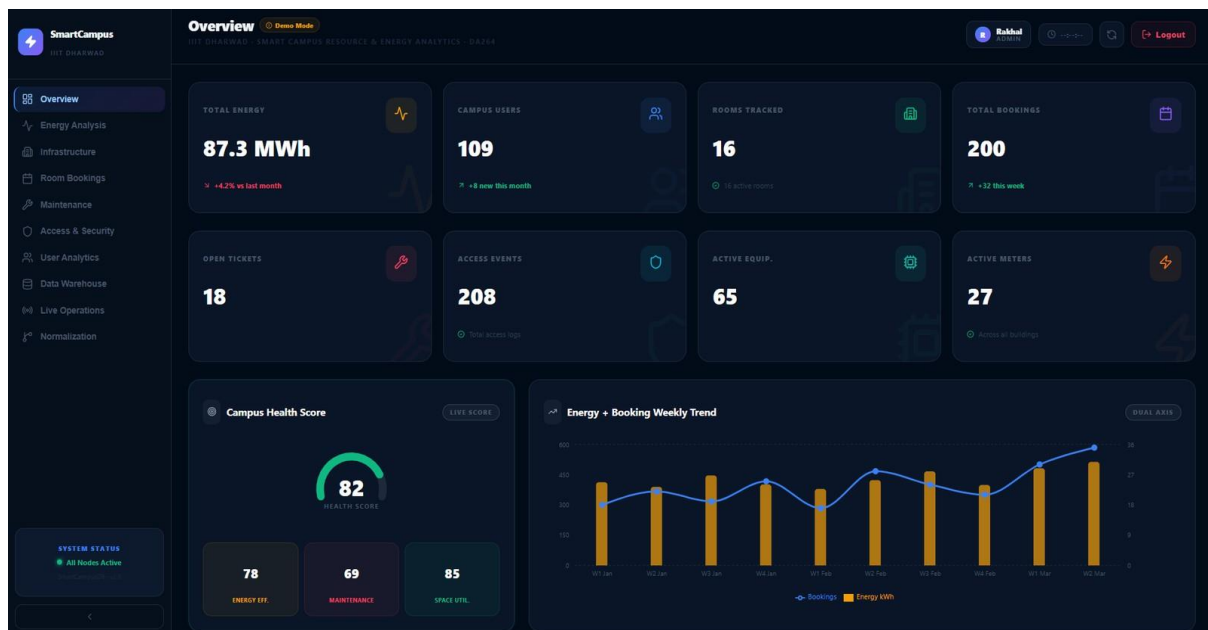


Figure 12.1a — Overview: Eight KPI Cards and Campus Health Score

The **Overview** page is the landing screen after login and serves as the campus command centre. The top row displays eight live KPI cards: **Total Energy** (87.3 MWh, +4.2% vs last month — aggregated from ENERGY_READING); **Campus Users** (109 registered users from USERS table); **Rooms Tracked** (16 active rooms from ROOM table); **Total Bookings** (200, +32 this week — COUNT from BOOKING table); **Open Tickets** (18 unresolved rows in MAINTENANCE_TICKET); **Access Events** (208 total ACCESS_LOG entries); **Active Equipment** (65 items with status = "Active" from EQUIPMENT); **Active Meters** (27 ENERGY_METER records across all buildings).

The **Campus Health Score** (82/100) is a composite live score derived from three weighted sub-scores: Energy Efficiency (78) measures proximity to consumption baselines; Maintenance Score (69) penalises open and SLA-breaching tickets; Space Utilization (85) measures booking density against room capacity. The gauge dial and sub-score cards refresh every 28 seconds.

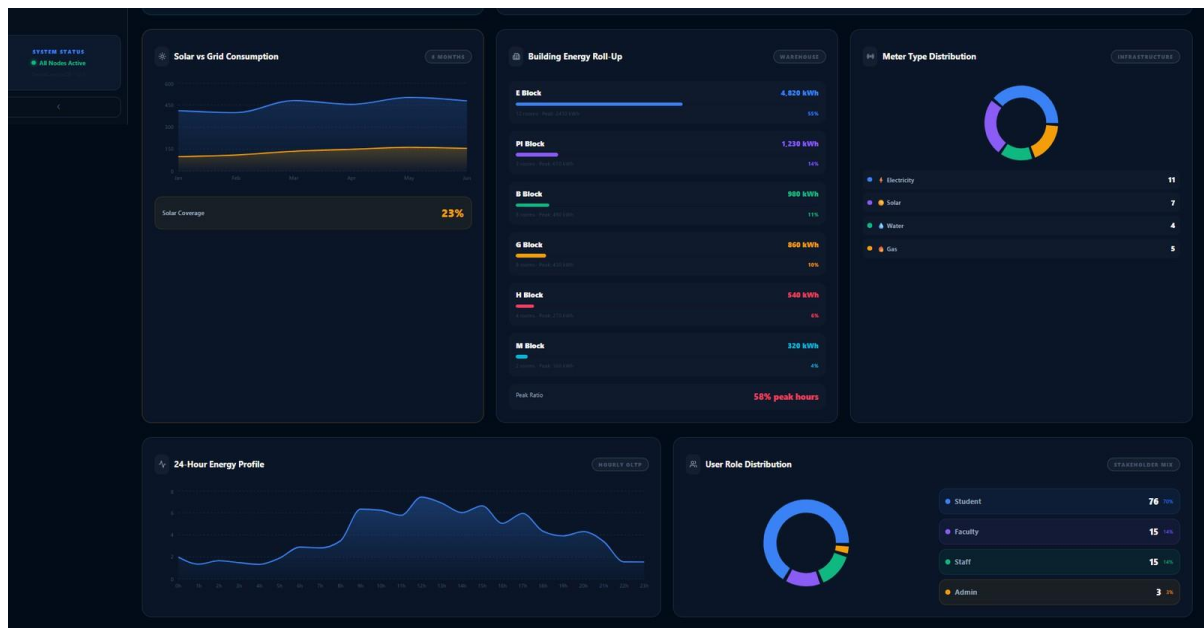


Figure 12.1b — Overview: Solar vs Grid, Building Energy Roll-Up, Meter Distribution, 24-Hour Profile

Lower panels: **Solar vs Grid Consumption** (area chart) — 6-month series comparing solar (orange, 23% coverage) vs grid (blue) energy from ENERGY_READING joined on meter_type. **Building Energy Roll-Up** (tagged "WAREHOUSE") — E Block 4,820 kWh (55%), PI Block 1,230 kWh (14%), B Block 980 kWh (11%), G Block 860 kWh (10%), H Block 540 kWh (6%), M Block 320 kWh (4%); peak ratio 58% — sourced from fact_energy_consumption joined with dim_room. **Meter Type Distribution** (donut) — 11 Electricity, 7 Solar, 4 Water, 5 Gas meters. **24-Hour Energy Profile** (line, tagged "HOURLY OLTP") — kWh by hour showing double-peak pattern (~9 AM class startup, ~3 PM afternoon peak). **User Role Distribution** (donut) — 76 Students (70%), 15 Faculty (14%), 15 Staff (14%), 3 Admins (3%).

12.2 Energy Analysis

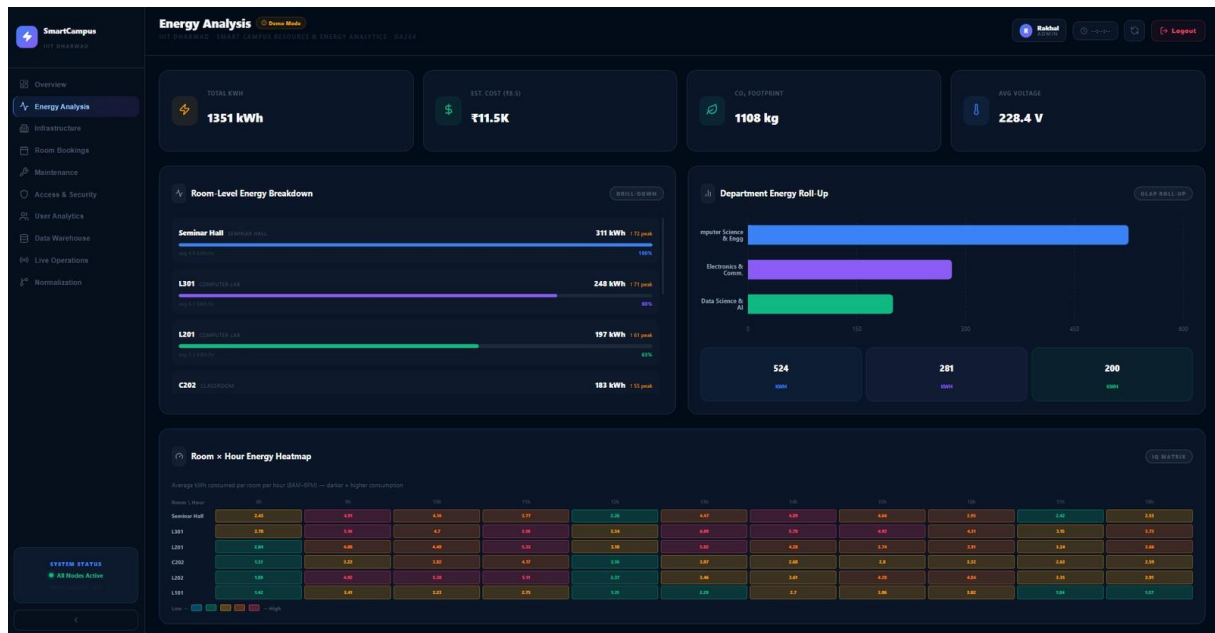


Figure 12.2a — Energy Analysis: KPIs, Room Breakdown, Department Roll-Up, Heatmap

Four top KPIs: **Total kWh** (1,351 kWh for current period); **Est. Cost** (₹11.5K at ₹8.5/unit); **CO₂ Footprint** (1,108 kg using 0.82 kg CO₂/kWh factor); **Avg Voltage** (228.4 V mean across active meters).

Room-Level Energy Breakdown (tagged "DRILL-DOWN") ranks rooms by kWh: Seminar Hall 311 kWh (100% baseline, peak 1.72x), L301 Computer Lab 248 kWh (80%, peak 1.71x), L201 197 kWh (63%), C202 183 kWh (55%). The peak multiplier is computed using the SQL LAG() window function comparing each reading against the previous one for the same meter.

Department Energy Roll-Up (tagged "OLAP ROLL-UP") uses GROUP BY ROLLUP to aggregate: CSE 524 kWh, ECE 281 kWh, DSAI 200 kWh. **Room x Hour Energy Heatmap** (tagged "IQ MATRIX") is a colour grid (8 AM–6 PM on x-axis, rooms on y-axis) where darker red = higher average kWh, sourced from an OLAP SLICE query on dim_date.hour BETWEEN 8 AND 18 grouped by room and hour.

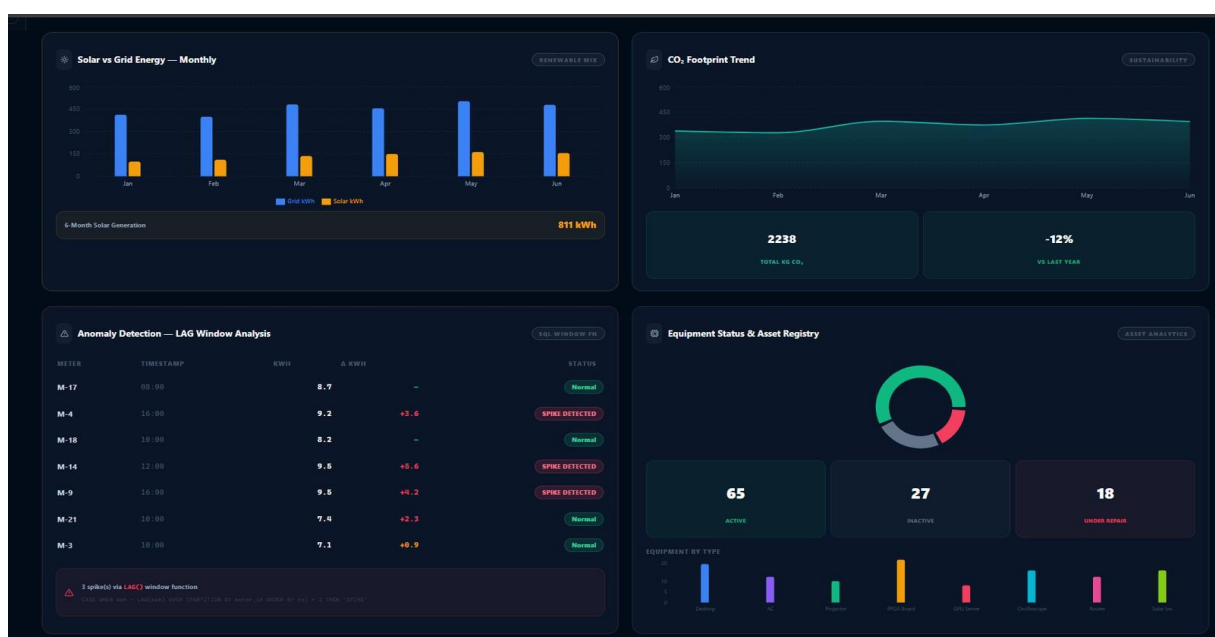


Figure 12.2b — Energy Analysis: Solar Monthly, CO₂ Trend, Anomaly Detection, Equipment Status

Solar vs Grid Energy Monthly (bar chart) — 811 kWh generated by solar over 6 months. **CO₂ Footprint Trend** — Total 2,238 kg CO₂, -12% vs last year. **Anomaly Detection — LAG Window Analysis** (tagged "SQL WINDOW FN") uses SQL Server's LAG() function: CASE WHEN kwh - LAG(kwh) OVER (PARTITION BY meter_id ORDER BY ts) > 2 THEN 'SPIKE'. Meters M-4 (+3.6), M-14 (+5.6), M-9 (+4.2) are currently flagged as SPIKE DETECTED. **Equipment Status** (donut + bar) — 65 Active, 27 Inactive, 18 Under Repair; broken down by type (Desktop, AC, Projector, FPGA Board, GPU Server, etc.) from EQUIPMENT joined with EQUIPMENT_TYPE.

12.3 Infrastructure

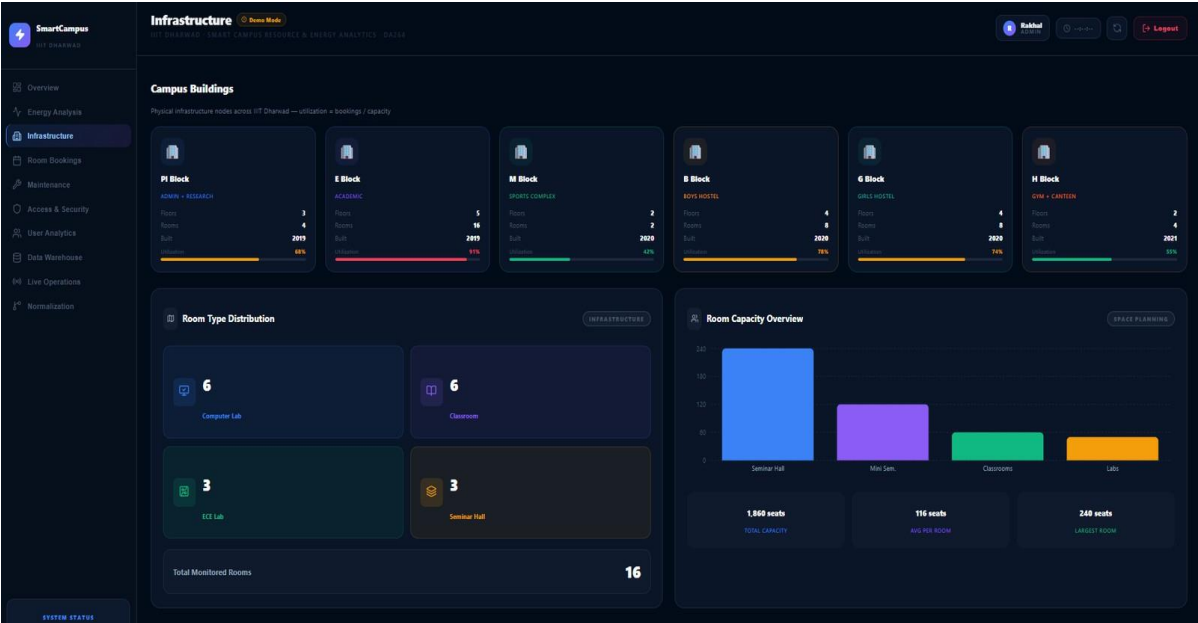


Figure 12.3a — Infrastructure: Campus Buildings and Room Type Distribution

Campus Buildings — one card per building, each showing floors, room count, build year, and utilization bar sourced from BUILDING joined with FLOOR and ROOM aggregations: PI Block (Admin+Research, 3 floors, 4 rooms, 68% util); E Block (Academic, 5 floors, 16 rooms, **91% util — highest**, red bar); M Block (Sports, 2 floors, 42%); B Block (Boys Hostel, 4 floors, 78%); G Block (Girls Hostel, 4 floors, 74%); H Block (Gym+Canteen, 2 floors, 55%). Utilization = (approved bookings this month) / (total bookable slots × room count).

Room Type Distribution — 6 Computer Labs, 6 Classrooms, 3 ECE Labs, 3 Seminar Halls across 16 total monitored rooms. **Room Capacity Overview** (tagged "SPACE PLANNING") — Total 1,860 seats, Average 116 seats/room, Largest Room 240 seats.

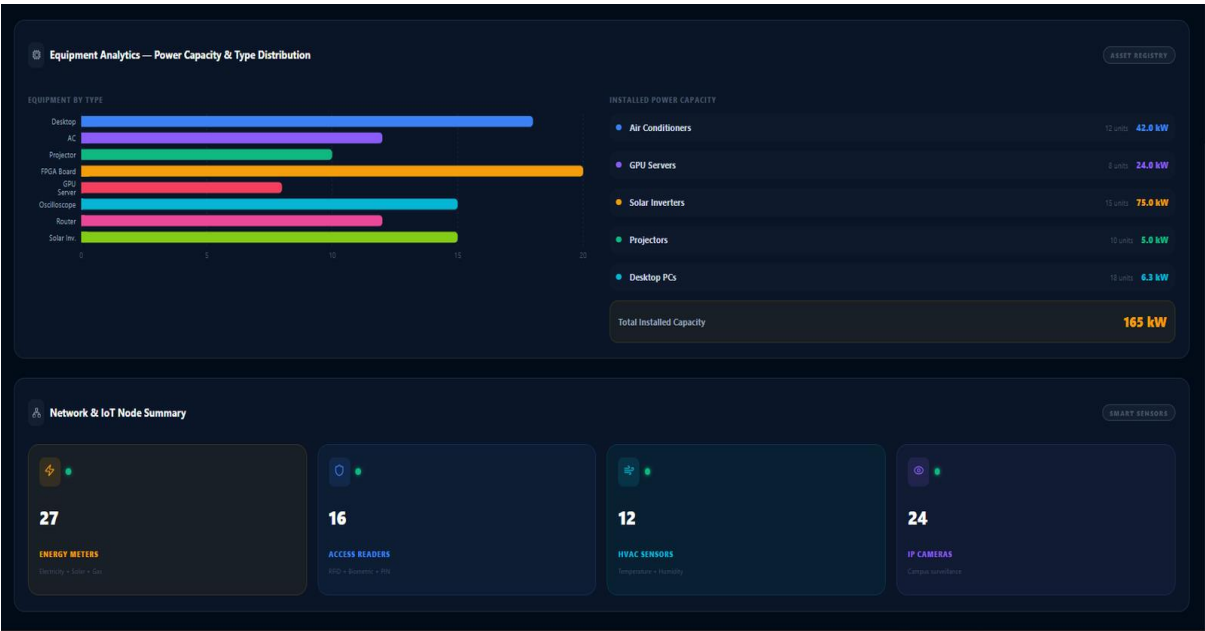


Figure 12.3b — Infrastructure: Equipment Analytics and IoT Node Summary

Equipment Analytics (tagged "ASSET REGISTRY") — horizontal bar by count (Desktop and FPGA Board most numerous) and installed power capacity: Air Conditioners 42.0 kW, GPU Servers 24.0 kW, Solar Inverters 75.0

kW, Projectors 5.0 kW, Desktop PCs 6.3 kW; **Total Installed Capacity = 165 kW. Network & IoT Node Summary** (tagged "SMART SENSORS") — 27 Energy Meters, 16 Access Readers (RFID+Biometric+PIN), 12 HVAC Sensors (Temp+Humidity), 24 IP Cameras — all showing green online status.

12.4 Room Bookings

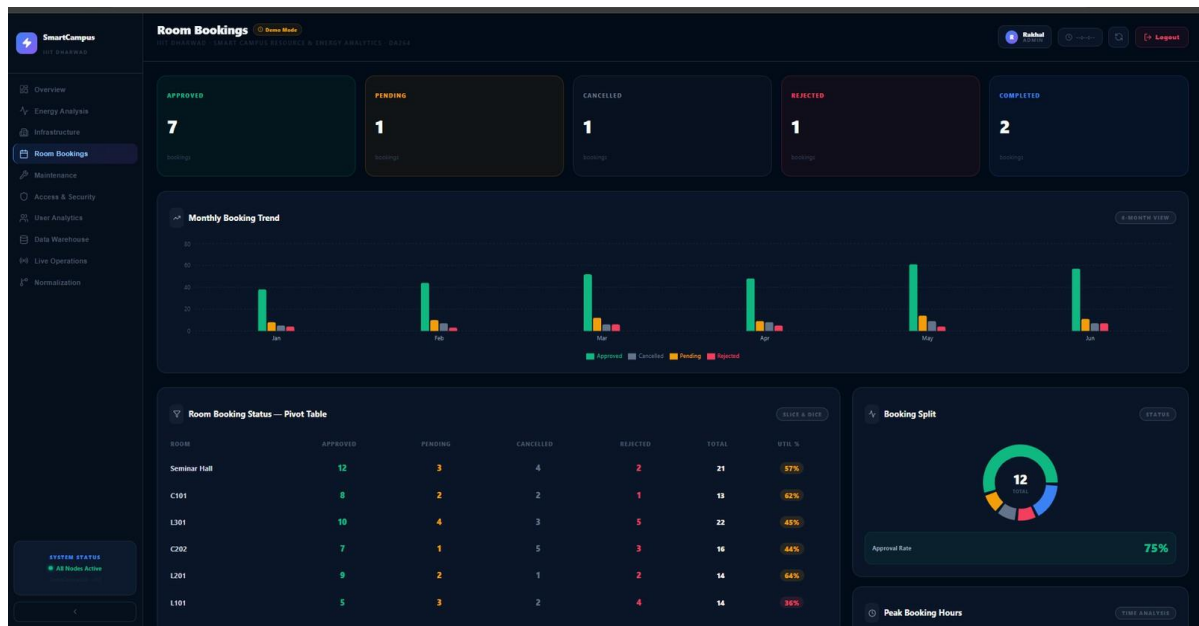


Figure 12.4a — Room Bookings: Status KPIs, Monthly Trend, Pivot Table, Booking Split

Five status KPI cards: Approved (7), Pending (1), Cancelled (1), Rejected (1), Completed (2) — from GROUP BY status COUNT on BOOKING filtered to current week. **Monthly Booking Trend** (6-month grouped bar) — Approved (green), Cancelled (grey), Pending (amber), Rejected (red) per month, showing approval patterns. **Room Booking Status Pivot Table** (tagged "SLICE & DICE") — implements SQL PIVOT cross-tabulating each room against status columns, with Total and Utilization %: L301 has highest total (22 bookings), C101 highest utilization (62%). **Booking Split** (donut) — 75% approval rate. **Peak Booking Hours** bar chart — shows demand by hour across the day.

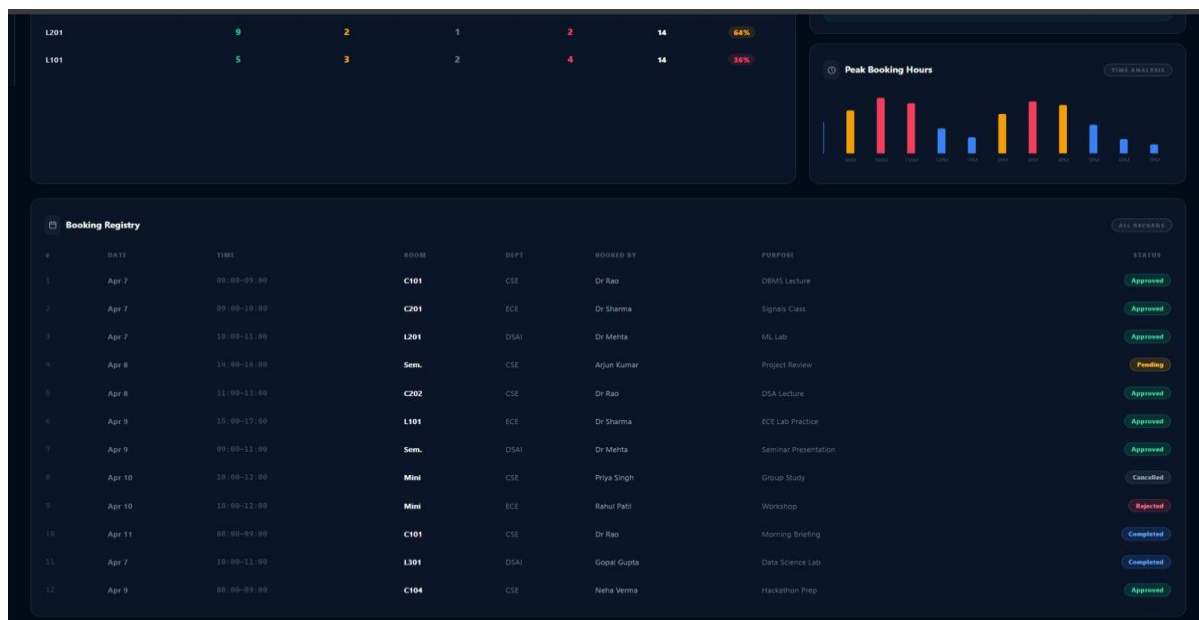


Figure 12.4b — Room Bookings: Full Booking Registry

Booking Registry (tagged "ALL RECORDS") lists every BOOKING row with: Date, Time, Room, Department, Booked By, Purpose, and colour-coded Status badge (Approved green, Pending amber, Rejected red, Cancelled grey, Completed blue). Sourced from BOOKING joined with USERS, ROOM, and DEPARTMENT ordered by date. Examples: Dr Rao → C101 (DBMS Lecture, Approved), Dr Sharma → C201 (Signals Class, Approved),

Arjun Kumar → Seminar Hall (Project Review, Pending).

12.5 Maintenance

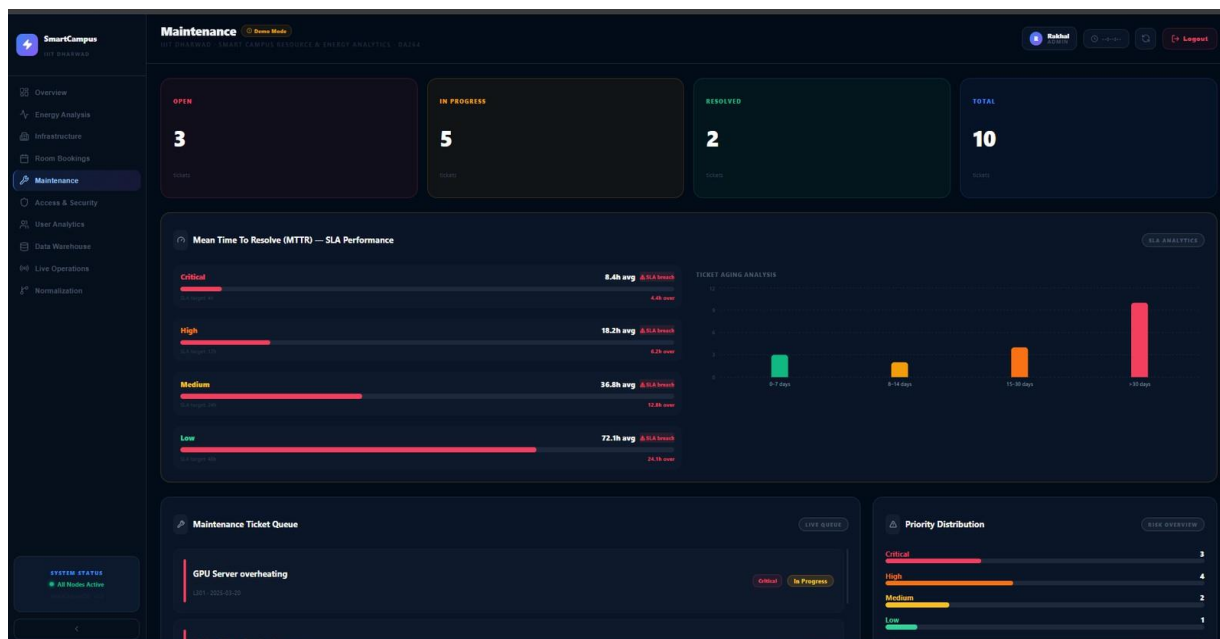


Figure 12.5a — Maintenance: Status KPIs, MTTR SLA Performance, Ticket Aging

KPI cards: Open (3), In Progress (5), Resolved (2), Total (10). **Mean Time To Resolve (MTTR) — SLA Performance** (tagged "SLA ANALYTICS") computes $AVG(DATEDIFF(hour, raised_at, resolved_at))$ per priority tier and compares against SLA targets — all four tiers are currently in breach: Critical 8.4h avg (target 4h, 4.4h over), High 18.2h (target 10h, 8.2h over), Medium 36.8h (target 24h, 12.8h over), Low 72.1h (target 48h, 24.1h over). **Ticket Aging Analysis** (bar) groups open tickets by age: 0–7 days (green), 8–14 days (amber), 15–30 days (orange), >30 days (red, largest bar).

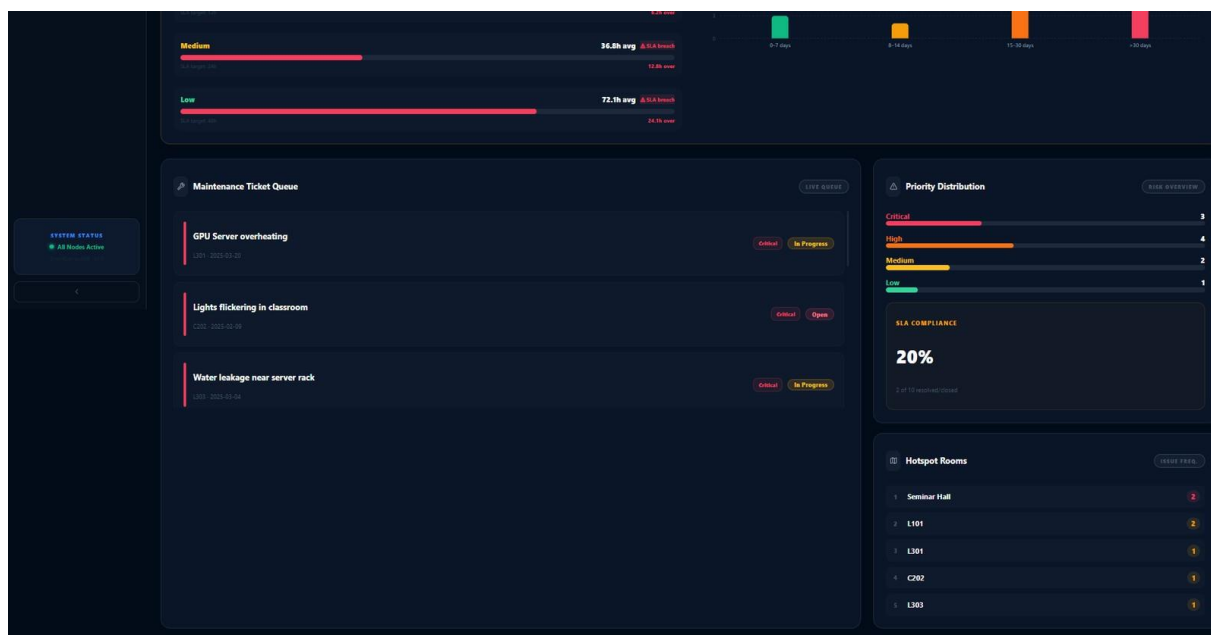


Figure 12.5b — Maintenance: Live Queue, Priority Distribution, SLA Compliance, Hotspot Rooms

Maintenance Ticket Queue (tagged "LIVE QUEUE") shows open/in-progress tickets as colour-coded cards: GPU Server overheating in L301 (Critical, In Progress); Lights flickering in C202 (Critical, Open); Water leakage near server rack in L303 (Critical, In Progress). **Priority Distribution** — 3 Critical, 4 High, 2 Medium, 1 Low. **SLA Compliance = 20%** (only 2 of 10 tickets resolved within SLA window). **Hotspot Rooms** (tagged "ISSUE FREQ") — Seminar Hall (2 tickets), L101 (2), L301 (1), C202 (1), L303 (1) — guides preventive maintenance priority.

12.6 Access & Security

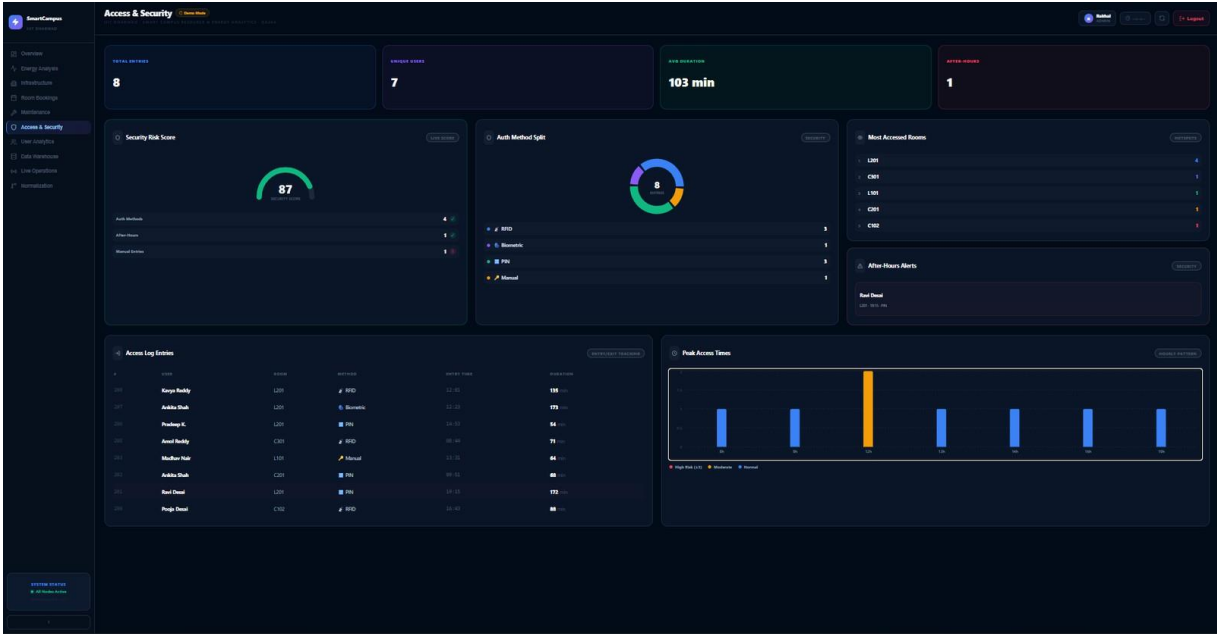


Figure 12.6 — Access & Security: Security Score, Auth Split, Access Logs, Peak Times

KPI cards: Total Entries (8), Unique Users (7), Avg Duration (103 min), After-Hours (1 flagged entry). **Security Risk Score** (radial gauge, 87/100, tagged "LIVE SCORE") — awards points for multi-factor auth usage, penalises after-hours manual entries and high-frequency access to sensitive rooms. Breakdown: Auth Methods used = 4 (positive), After-Hours = 1 (warning), Manual Entries = 1 (flagged red). **Auth Method Split** (donut) — RFID (3), Biometric (1), PIN (3), Manual (1). **Most Accessed Rooms** — L201 (4 entries), C301, L101, C201, C102 (1 each). **After-Hours Alerts** — Ravi Desai late-night L201 entry flagged. **Access Log Entries** table (tagged "ANTI-SUIT TRACKING") — lists each log row: user, room, auth method, time, duration. **Peak Access Times** (bar chart) — 12h noon highlighted orange as high-risk simultaneous-access hour; other hours blue (normal).

12.7 User Analytics

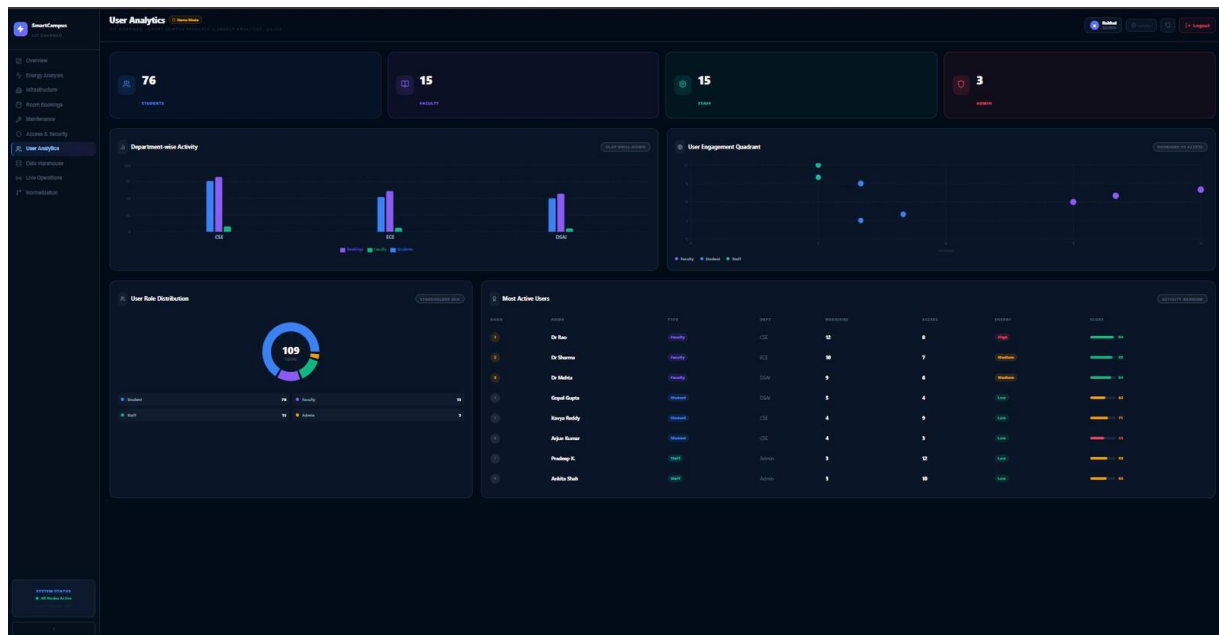


Figure 12.7 — User Analytics: Role Distribution, Department Activity, Engagement Quadrant, Leaderboard

KPI cards: 76 Students, 15 Faculty, 15 Staff, 3 Admins (total 109). **Department-wise Activity** (grouped bar, tagged "OLAP DRILL-DOWN") — compares bookings, faculty logins, and student access across CSE, ECE, DSAI from a JOIN across BOOKING, ACCESS_LOG, USERS, and DEPARTMENT. **User Engagement Quadrant** (scatter, tagged "BOOKINGS VS ACCESS") — each user plotted as a point: x = bookings, y = access events. Faculty cluster toward high-booking/high-access; students spread widely; staff typically low-booking/high-access. **User Role Distribution** (donut) — proportional arcs for all four roles.

Most Active Users leaderboard (tagged "ACTIVITY RANKING") — composite score = $0.4 \times \text{bookings} + 0.3 \times \text{access} + 0.3 \times \text{energy_rank}$: Dr Rao (Faculty, CSE) — 12 bookings, 8 access, High energy, score 94; Dr Sharma (ECE) — 10 bookings, 7 access, score 88; Dr Mehta (DSAI) — 9 bookings, 6 access, score 84; Student Gopal Gupta (DSAI) — 5 bookings, 4 access, score 62.

12.8 Data Warehouse

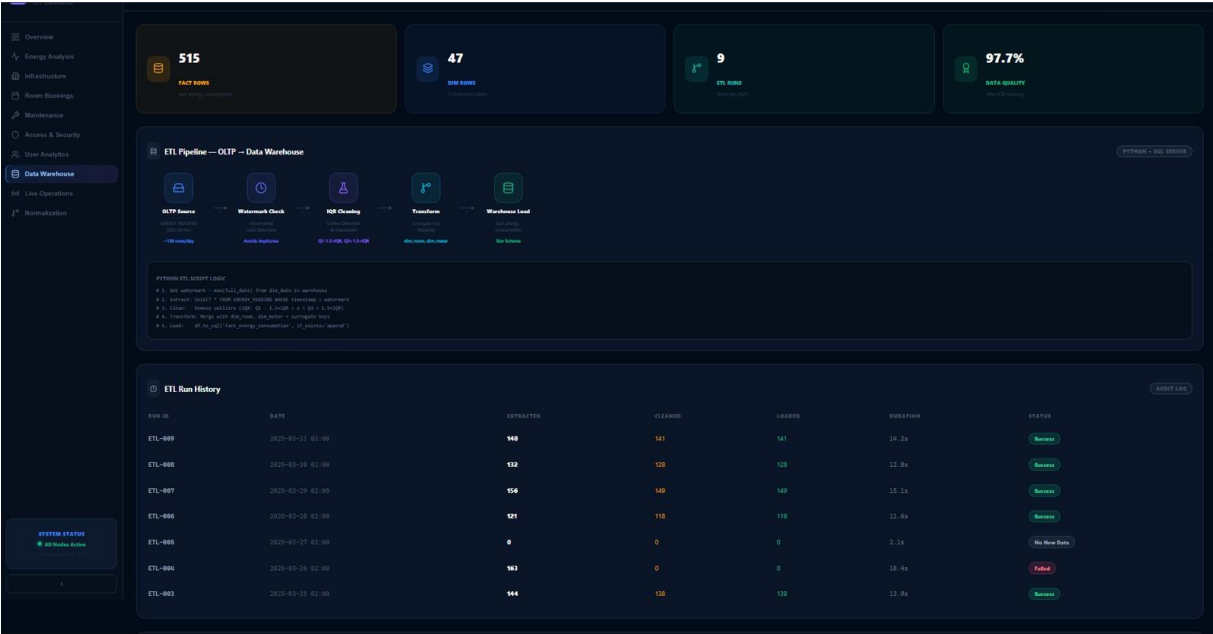


Figure 12.8a — Data Warehouse: ETL Pipeline Flow and Run History

KPI cards: Fact Rows (515), Dim Rows (47), ETL Runs (9 since Jan 2025), Data Quality (97.7% — rows passing IQR filter). **ETL Pipeline — OLTP → Data Warehouse** (tagged "PYTHON + SQL SERVER") visualises the 5-step flow: OLTP Source → Watermark Check (avoids duplicates) → IQR Cleaning ($Q1-1.5 \times IQR$ to $Q3+1.5 \times IQR$) → Transform (surrogate key mapping via `dim_room`, `dim_meter`) → Warehouse Load (`fact_energy_consumption`, Star Schema). The Python ETL script logic is shown verbatim in a code panel.

ETL Run History (tagged "AUDIT LOG") — ETL-009 (148 extracted, 141 loaded, 14.2s, Success); ETL-008 (132→128, Success); ETL-007 (156→149, Success); ETL-005 (0 extracted, "No New Data" — watermark matched); ETL-004 (163 extracted, 0 loaded, "Failed" — IQR rejected all rows as outliers). This audit trail demonstrates incremental load design and data quality enforcement.

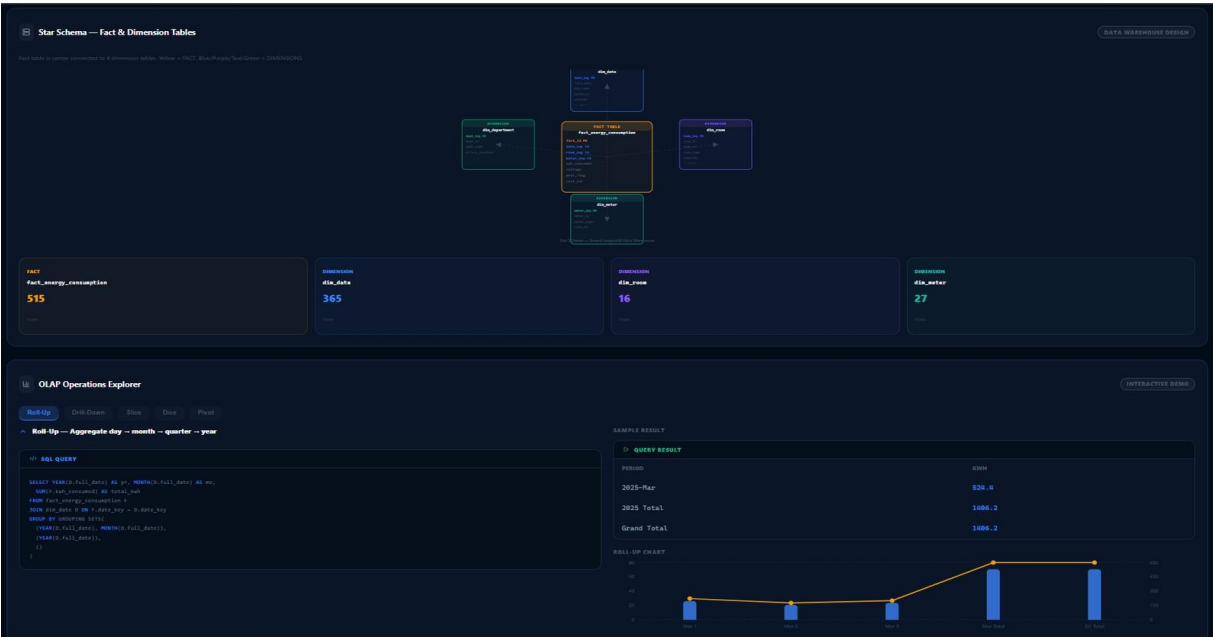


Figure 12.8b — Data Warehouse: Star Schema Diagram and OLAP Operations Explorer

Star Schema (tagged "DATA WAREHOUSE DESIGN") — visual miniature of fact_energy_consumption (central, 515 rows) connected to dim_date (365), dim_room (16), dim_meter (27), and dim_department (Snowflake). **OLAP Operations Explorer** (5 tabs: Roll-Up, Drill-Down, Slice, Dice, Pivot) — each tab shows the actual SQL query and a sample result. Roll-Up tab shows GROUP BY GROUPING SETS aggregating day→month→quarter→year with results: 2025-Mar = 524.8 kWh, 2025 Total = 1,406.2 kWh, rendered alongside a Roll-Up chart.

12.9 Normalization

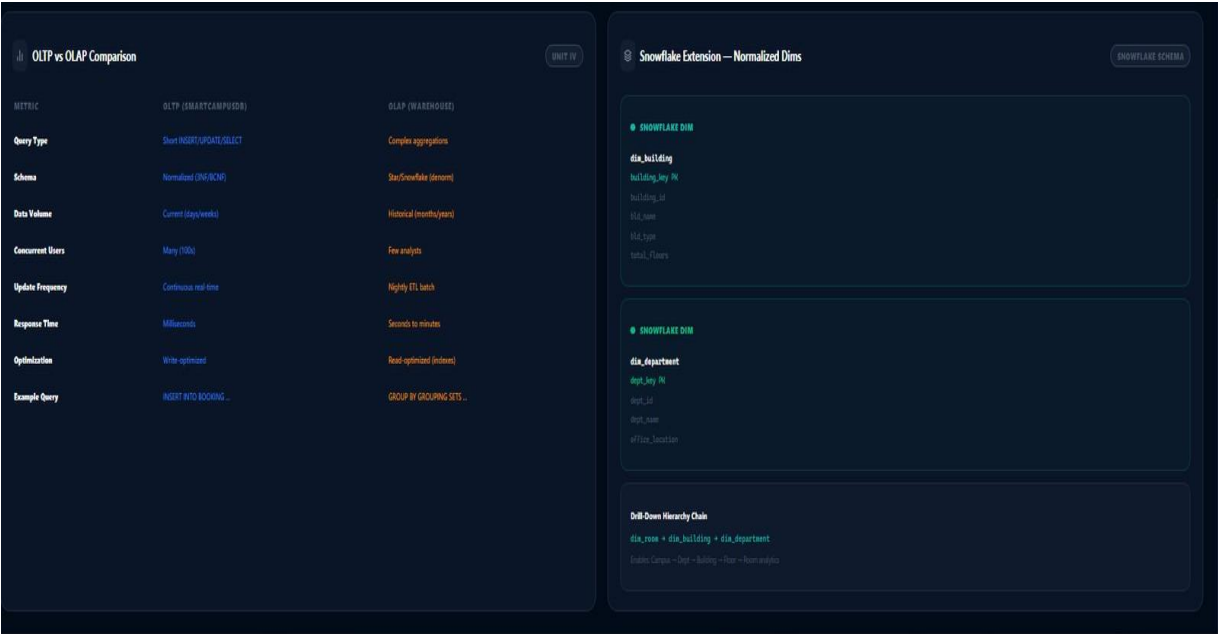


Figure 12.9 — Normalization: OLTP vs OLAP Comparison and Snowflake Extension

Metric	OLTP (SmartCampusDB)	OLAP (Warehouse)
Query Type	Short INSERT/UPDATE/SELECT	Complex aggregations
Schema	Normalized 3NF/BCNF	Star / Snowflake (denorm)
Data Volume	Current days/weeks	Historical months/years
Concurrent Users	Many (100s)	Few analysts
Update Frequency	Continuous real-time	Nightly ETL batch
Response Time	Milliseconds	Seconds to minutes
Optimization	Write-optimized	Read-optimized (indexes)
Example Query	INSERT INTO BOOKING ...	GROUP BY GROUPING SETS ...

Snowflake Extension — Normalized Dims: dim_room references dim_building (building_key PK, building_id, bld_name, bld_type, total_floors) which references dim_department (dept_key PK, dept_id, dept_name, office_location). The drill-down hierarchy chain: dim_room → dim_building → dim_department enables Campus → Dept → Building → Floor → Room level OLAP analytics.

13. Live Operations — Complete Guide

Figure 13.1 — Live Operations Center: Four Real-Time Data Entry Forms + ETL Trigger

The **Live Operations** page is the real-time data entry console of the SmartCampus dashboard. Every action here executes a direct SQL INSERT into the SmartCampusDB OLTP tables via the FastAPI backend. The page header shows a **28-second auto-refresh countdown** and a green "Live" indicator, confirming the dashboard re-queries the database at that interval. The four top KPI counters — Energy Readings, Total Bookings, Open Tickets, Access Events — update after every successful form submission.

13.1 Log Energy Reading

This form inserts a new row into the **ENERGY_READING** table. Fields: **Meter** (dropdown from SELECT meter_id, meter_type FROM ENERGY_METER), **kWh Consumed** (numeric, e.g. 4.75), **Voltage (V)** (pre-filled to 230 — Indian standard), **Peak Hour** (Yes/No toggle → peak_flag BIT column).

How to fill the form:

- Select a meter from the dropdown — e.g. M-4 (Electricity, Room L301).
- Enter kWh Consumed — e.g. 9.2 (a spike value for demonstration).
- Leave Voltage at 230 V unless you are simulating a voltage anomaly.
- Set Peak Hour to "Yes" if simulating a 9 AM–12 PM or 2 PM–5 PM reading.
- Click "Insert Reading → SQL Server".

SQL that executes on the backend:

```
INSERT INTO ENERGY_READING (meter_id, timestamp, kwh_consumed, voltage, peak_flag) VALUES (?,  
GETDATE(), ?, ?, ?)
```

The **reading_id** is an IDENTITY column (auto-increments). The timestamp is set to SQL Server's current datetime via GETDATE(). An **AFTER INSERT trigger** on ENERGY_READING immediately writes an AUDIT_TRAIL entry recording the table name, operation = "INSERT", changed_by, and changed_at.

How to verify in SSMS:

- SELECT TOP 1 * FROM ENERGY_READING ORDER BY reading_id DESC — your new row should appear.

- `SELECT TOP 1 * FROM AUDIT_TRAIL ORDER BY audit_id DESC` — trigger entry should appear.

Dashboard visuals that change after this insert:

- Overview → Total Energy KPI card: cumulative kWh increases.
- Overview → 24-Hour Energy Profile: current hour bar rises.
- Overview → Building Energy Roll-Up: the relevant building's kWh share increases.
- Energy Analysis → Room-Level Energy Breakdown: that room may move up the ranking.
- Energy Analysis → Anomaly Detection LAG panel: if delta vs previous reading exceeds threshold, row appears with "SPIKE DETECTED" in red (e.g., M-4 +3.6 delta).
- Energy Analysis → Room × Hour Heatmap: cell for (this room, current hour) becomes darker.
- Data Warehouse (after ETL) → Fact Rows counter increases; Building Roll-Up and Dept Roll-Up update from the refreshed warehouse.

13.2 Create Room Booking

This form inserts a new row into the **BOOKING** table. Fields: **User** (dropdown from USERS), **Room** (dropdown from ROOM), **Start Time** (datetime picker), **End Time** (datetime picker), **Purpose** (free text, e.g. "DBMS Lab Session").

How to fill the form:

- Select User — e.g. Dr Rao (Faculty, CSE).
- Select Room — e.g. L301 (Computer Lab).
- Set Start Time: 2025-04-05 10:00 and End Time: 2025-04-05 12:00.
- Enter Purpose: "ML Lab Practice".
- Click "Submit Booking → SQL Server".

SQL that executes on the backend:

```
INSERT INTO BOOKING (user_id, room_id, start_time, end_time, purpose, status) VALUES (?, ?, ?, ?, ?, 'Pending')
```

New bookings always start with **status = 'Pending'**. A stored procedure checks for time-slot conflicts: if another APPROVED booking already exists for the same room overlapping the same window, the insert is rolled back and an error is returned. If no conflict, the booking commits and an AUDIT_TRAIL trigger entry is generated.

How to verify in SSMS:

- `SELECT TOP 1 * FROM BOOKING ORDER BY booking_id DESC` — new row with status = 'Pending'.
- `SELECT * FROM BOOKING WHERE room_id = ? AND status = 'Pending'` — all pending for that room.

Dashboard visuals that change:

- Overview → Total Bookings KPI: count +1; "+1 this week" delta increments.
- Room Bookings → Pending KPI card: increases by 1.
- Room Bookings → Monthly Booking Trend: current month amber (Pending) bar grows.
- Room Bookings → Room Booking Status Pivot Table: that room's Pending and Total columns increment.
- Room Bookings → Booking Registry: new row at top with "Pending" amber badge.
- Infrastructure → Building Utilization bar: if the room's building gains a new approved booking later, utilization % updates.
- User Analytics → Most Active Users leaderboard: booker's "Bookings" count increments, affecting their composite score.

13.3 Raise Maintenance Ticket

This form inserts a row into the **MAINTENANCE_TICKET** table (the weak entity). Fields: **Room** (dropdown from ROOM), **Reported By** (dropdown from USERS), **Priority** (Critical / High / Medium / Low), **Description** (free text describing the issue).

How to fill the form:

- Select Room — e.g. L301 (Computer Lab).
- Select Reported By — e.g. Pradeep K (Staff).
- Set Priority to Critical.
- Enter Description: "Air conditioner not working, room temperature above 32°C".
- Click "Raise Ticket → SQL Server".

SQL that executes on the backend:

```
INSERT INTO MAINTENANCE_TICKET (room_id, reported_by, issue_desc, priority, status, raised_at)
VALUES (?, ?, ?, ?, 'Open', GETDATE())
```

Status is always **'Open'** on creation. Because MAINTENANCE_TICKET is a **weak entity**, room_id cannot be NULL — an invalid room_id triggers a FK constraint violation and the insert is rejected. The MTTR clock starts from raised_at. An AUDIT_TRAIL trigger fires on insert.

How to verify in SSMS:

- SELECT TOP 1 * FROM MAINTENANCE_TICKET ORDER BY ticket_id DESC — new row with status = 'Open'.
- SELECT COUNT(*) FROM MAINTENANCE_TICKET WHERE status = 'Open' — should be 1 more than before.

Dashboard visuals that change:

- Overview → Open Tickets KPI: increments by 1.
- Overview → Campus Health Score: Maintenance sub-score (69) drops as open ticket count grows.
- Maintenance → Open KPI card and Total KPI card: both increment.
- Maintenance → Maintenance Ticket Queue (LIVE QUEUE): new card at top with red Critical badge and Open status.
- Maintenance → Priority Distribution bar: Critical bar grows from 3 → 4.
- Maintenance → SLA Compliance %: decreases as denominator (total tickets) grows without resolved count changing.
- Maintenance → Ticket Aging Analysis: new ticket appears in the 0–7 days (green) bar; shifts right over time if unresolved.
- Maintenance → Hotspot Rooms: if that room already has tickets, its count increments; if new, it enters the ranked list.
- Access & Security → Security Risk Score: sensitive lab tickets may lower the score.

13.4 Log Access Event

This form inserts a row into the **ACCESS_LOG** table. Fields: **User** (dropdown from USERS), **Room** (dropdown from ROOM), **Auth Method** (dropdown: RFID / Biometric / PIN / Manual).

How to fill the form:

- Select User — e.g. Kavya Reddy (Student, CSE).
- Select Room — e.g. L201.
- Select Auth Method: RFID.
- Click "Log Access → SQL Server".

SQL that executes on the backend:

```
INSERT INTO ACCESS_LOG (user_id, room_id, entry_time, auth_method) VALUES (?, ?, GETDATE(), ?)
```

entry_time is set to the current SQL Server timestamp. duration_min is NULL at insert (updated when the user exits). If entry_time falls outside campus hours (after 10 PM), it is flagged as an **after-hours entry** and immediately appears in the After-Hours Alerts panel on Access & Security.

How to verify in SSMS:

- SELECT TOP 1 * FROM ACCESS_LOG ORDER BY log_id DESC — confirms new row with current entry_time.
- SELECT COUNT(*) FROM ACCESS_LOG WHERE CAST(entry_time AS DATE) = CAST(GETDATE() AS DATE) — today's total entries.

Dashboard visuals that change:

- Overview → Access Events KPI: increments (e.g. 208 → 209).
- Access & Security → Total Entries KPI: increments.
- Access & Security → Unique Users KPI: increments if this user had no prior entry today.
- Access & Security → Auth Method Split donut: chosen method slice grows.
- Access & Security → Most Accessed Rooms: L201 count increments; ranking may shift.
- Access & Security → Access Log Entries table: new row appears at top.
- Access & Security → Peak Access Times chart: current hour bar grows.
- Access & Security → Security Risk Score: decreases if auth_method = "Manual"; unchanged for RFID/Biometric.
- After-hours entry → After-Hours KPI increments; user name appears in After-Hours Alerts panel.
- User Analytics → Most Active Users: this user's Access count increments, affecting composite score and rank.

13.5 Trigger ETL Pipeline

The green "■ Run ETL Now" button runs `etl_pipeline.py` to sync new OLTP ENERGY_READING rows into the fact_energy_consumption OLAP warehouse table. Use this after inserting a batch of energy readings via "Log Energy Reading".

Step-by-step pipeline execution:

- Step 1 — Watermark Check: SELECT MAX(full_date) FROM dim_date WHERE date_key IN (SELECT DISTINCT date_key FROM fact_energy_consumption). Returns last loaded date, e.g. 2025-03-31.
- Step 2 — Extract: SELECT reading_id, timestamp, kwh_consumed, voltage, peak_flag, meter_id FROM ENERGY_READING WHERE timestamp > '2025-03-31'. Pulls only new rows.
- Step 3 — IQR Cleaning: Q1 and Q3 of kwh_consumed computed; readings outside [Q1 – 1.5×IQR, Q3 + 1.5×IQR] are discarded as sensor errors. Count of removed outliers is logged.
- Step 4 — Transform: room_id and meter_id joined to dim_room and dim_meter to obtain surrogate keys room_key and meter_key. date_key computed as YYYYMMDD integer.
- Step 5 — Load: df.to_sql('fact_energy_consumption', engine, if_exists='append', index=False) bulk-inserts the cleaned rows.

How to verify in SSMS:

- SELECT COUNT(*) FROM fact_energy_consumption — should be higher than before.
- SELECT TOP 5 * FROM fact_energy_consumption ORDER BY fact_id DESC — your new readings appear.
- Data Warehouse page → ETL Run History: a new ETL-0XX row appears at top with Extracted, Cleaned, Loaded counts and duration.

Dashboard visuals that change:

- Data Warehouse → Fact Rows KPI: increases by rows loaded.
- Data Warehouse → Data Quality %: recalculates (loaded / extracted × 100).
- Data Warehouse → ETL Run History: new run row at top (Success or No New Data or Failed).
- Overview → Building Energy Roll-Up (WAREHOUSE tag): updates as warehouse aggregation refreshes.
- Energy Analysis → Department Energy Roll-Up: refreshes from updated fact table.
- Data Warehouse → OLAP Operations Explorer: Roll-Up and Drill-Down tabs reflect latest fact data.

13.6 End-to-End Demonstration Sequence

The following 7-step sequence exercises every form and traces data flow across the full stack — from live UI input through OLTP SQL Server to OLAP warehouse and back to dashboard visuals.

Step 1 — Login: Log in as *rakhal* (Admin). Overview page loads with all 8 KPI cards showing current database state.

Step 2 — Insert Energy Reading: Live Operations → Log Energy Reading: select Meter M-4, kWh = 9.2, Voltage = 230, Peak = Yes → click Insert. Immediately check Energy Analysis → Anomaly Detection — M-4 should appear as "SPIKE DETECTED" (+3.6 delta vs previous reading).

Step 3 — Create Booking: Live Operations → Create Room Booking: User = Dr Rao, Room = L301, Start = 10:00, End = 12:00, Purpose = "DBMS Lab" → Submit. Navigate to Room Bookings — new "Pending" row appears in Booking Registry; Pending KPI card increments.

Step 4 — Raise Ticket: Live Operations → Raise Maintenance Ticket: Room = L301, Priority = Critical, Description = "Projector bulb blown" → Raise. Maintenance page → Open card rises; new Critical card appears in Live Queue; SLA Compliance % drops.

Step 5 — Log Access: Live Operations → Log Access Event: User = Kavya Reddy, Room = L201, Auth = RFID → Log. Access & Security → Total Entries increments; new row in Access Log table; RFID slice of Auth Method donut grows.

Step 6 — Run ETL: Click "■ Run ETL Now" → wait ~15 seconds. Data Warehouse → Fact Rows count increases; ETL Run History shows new Success row with extracted/loaded counts. Building Roll-Up on Overview refreshes with new warehouse data.

Step 7 — Check Overview: Return to Overview — all 8 KPI cards reflect the inserted data. Campus Health Score may shift downward from the new open ticket, and Building Energy Roll-Up shows the updated kWh for L301's building.

Conclusion

The systematic design of the SmartCampus Resource & Energy Analytics System resulted in a robust, fully-normalized OLTP database and a well-structured OLAP data warehouse, together forming a complete end-to-end DBMS project that demonstrates mastery of all four DA264 syllabus units.

Through the formal application of normalization rules from 1NF up to Boyce-Codd Normal Form (BCNF) across all 12 OLTP entities, every transitive dependency and redundancy was eliminated. Functional dependencies are documented both in the SQL schema (as FD comments above each CREATE TABLE) and visually in the interactive Normalization Dashboard built within the React frontend.

Key structural decisions — such as the Building → Floor → Room → Meter → Reading hierarchy to eliminate transitive chains, the EQUIPMENT_TYPE extraction to prevent deletion anomalies, and the AUDIT_TRAIL trigger system for automatic accountability — all contribute to a database that genuinely prevents insert, update, and delete anomalies rather than merely describing them.

The transaction management section demonstrated all four ACID properties with live SQL Server demos including a 7-minute isolation blocking scenario, a Msg 1205 deadlock event, and explicit Two-Phase Locking using HOLDLOCK and UPDLOCK hints. The data warehouse implements both Star and Snowflake schemas with all five OLAP operations (Roll-up, Drill-down, Slice, Dice, Pivot) and an incremental Python ETL pipeline with IQR-based data cleaning.

This solid conceptual and technical foundation makes the SmartCampus system a complete reference implementation for all learning outcomes in DA264.

References

The following external resources and documentation were instrumental in shaping the conceptual and theoretical foundations of this database design:

1. Ramakrishnan, R. & Gehrke, J. (2003). Database Management Systems (3rd ed.). McGraw-Hill. — Primary textbook for normalization theory, ER modeling, and transaction management.
2. Microsoft SQL Server Documentation. Transactions (Transact-SQL). <https://docs.microsoft.com/en-us/sql/t-sql/language-elements/transactions-transact-sql>
3. Microsoft SQL Server Documentation. SET TRANSACTION ISOLATION LEVEL. <https://docs.microsoft.com/en-us/sql/t-sql/statements/set-transaction-isolation-level-transact-sql>
4. Kimball, R. & Ross, M. (2013). The Data Warehouse Toolkit (3rd ed.). Wiley. — Reference for Star Schema and Snowflake schema design.
5. GeeksforGeeks. Normal Forms in DBMS. <https://www.geeksforgeeks.org/normal-forms-in-dbms/>
6. GeeksforGeeks. ACID Properties in DBMS. <https://www.geeksforgeeks.org/acid-properties-in-dbms/>
7. GeeksforGeeks. Two Phase Locking (2-PL) in DBMS. <https://www.geeksforgeeks.org/two-phase-locking-2-pl/>
8. GeeksforGeeks. Functional Dependency and Attribute Closure. <https://www.geeksforgeeks.org/dbms/functional-dependency-and-attribute-closure/>
9. FastAPI Documentation. <https://fastapi.tiangolo.com/> — Used for the Python REST API layer.
10. SQLAlchemy Documentation. <https://docs.sqlalchemy.org/> — Used for ETL pipeline database connectivity.
11. GeeksforGeeks. Indexing in Databases. <https://www.geeksforgeeks.org/dbms/indexing-in-databases-set-1/>
12. Microsoft SQL Server. sys.dm_exec_requests (Transact-SQL). <https://docs.microsoft.com/en-us/sql/relational-databases/system-dynamic-management-views/sys-dm-exec-requests-transact-sql>